

# DriftDroid: Temporal Drift-Aware Android Malware Detection Using Dynamic Behavioral Features

Muhammet Tan

**Abstract**—Machine learning-based Android malware detectors often report high accuracy under random or static evaluation protocols, yet deployed detectors must classify applications that appear after the training period and may follow different behavioral distributions. This paper reframes Android malware detection as a temporal generalization problem. We build a dynamic analysis pipeline that executes historical KronoDroid applications and recent AndroZoo samples in a controlled emulator environment, extracts sequential runtime behavioral tags together with aggregate dynamic features, and evaluates models under a strict train-past/test-future protocol. Beyond detection accuracy, the study quantifies behavioral drift between historical and recent samples using distributional metrics and feature-level interpretation. In particular, we analyze runtime behavioral tags whose prevalence changes substantially across time, such as features that rise from 20% in historical data to 65% in recent samples, to expose how malware behavior evolves and why static evaluation can overestimate real-world robustness.

**Index Terms**—Android malware, temporal drift, dynamic analysis, behavioral sequence, concept drift, realistic evaluation

## I. INTRODUCTION

Mobile devices have become indispensable to daily life, with Android commanding approximately 74% of the global mobile operating system market [1]. This dominance, coupled with the platform’s open-source architecture, has made Android a primary target for malicious actors. According to Kaspersky’s 2023 Mobile Threat Report, over 33.8 million malware, adware, and riskware attacks were blocked on mobile devices, representing a 50% year-over-year increase. More than 1.3 million unique malicious installation packages were identified, of which 154,000 contained mobile banking Trojans. These escalating figures underscore the urgent need for robust and adaptive malware detection mechanisms capable of keeping pace with an increasingly sophisticated threat landscape. The central difficulty is not only whether a model can separate malicious and benign applications in a curated benchmark, but whether it remains reliable when the Android ecosystem, malware families, platform APIs, and user-facing application behavior evolve after deployment.

A substantial body of work has addressed Android malware detection through classical machine learning and deep learning. Static detectors use permissions, API calls, intents, manifest declarations, opcodes, or bytecode-derived representations to provide efficient pre-execution screening [2]–[4]. Dynamic detectors execute applications and monitor runtime behavior such as system calls, API events, and network traffic, thereby capturing behaviors that may be hidden from static

inspection [5]–[8]. Hybrid detectors combine static and dynamic evidence to improve coverage and robustness [9]–[11]. These lines of work have produced strong reported scores, but many studies still evaluate models using random splits or temporally mixed datasets, allowing future-like samples to influence training and validation. Under such protocols, high accuracy may reflect benchmark convenience rather than deployment readiness.

Recent research has shown that this evaluation gap is not a minor methodological detail. Early empirical studies warned that Android malware evaluation must respect time when measuring generalization [12], and TESSERACT later demonstrated that spatial and temporal bias can substantially inflate malware-classification performance when train/test splits do not match realistic deployment constraints [13], [14]. KronoDroid was created specifically to support time-aware Android malware research by providing timestamped hybrid features over a broad historical period [15]. More recent surveys argue that Android malware detection is far from “mission accomplished”, identifying outdated datasets, neglected concept drift, dynamic-data consistency, model security, and explainability as unresolved obstacles for long-term detectors [16].

However, existing studies either focus on evaluation protocols or model adaptation, while lacking an explicit analysis of how runtime behaviors evolve over time and how such evolution affects detection performance. This paper therefore shifts the focus from simply proposing another dynamic detector to studying temporal drift in Android runtime behavior. We train on historical KronoDroid samples and reserve recent AndroZoo samples dated 2025–2026 exclusively for temporal testing [17]. Each application is executed in a controlled Genymotion-based environment, instrumented at runtime, and represented through both sequential behavioral tags and aggregate dynamic features. The objective is twofold: first, to measure how well sequence and hybrid models generalize from past to future samples; second, to explain how runtime behavior changes across time. For example, we explicitly examine behavioral tags whose prevalence increases from 20% in historical data to 65% in recent samples, since such shifts provide measurable evidence of malware evolution rather than only a drop in classifier accuracy.

### A. Contributions

The principal contributions of this work are summarized as follows:

Unlike prior works that treat temporal shift as an evaluation artifact, this study explicitly models and explains behavioral drift in dynamic Android malware detection.

- We formulate Android malware detection as a temporal generalization problem and evaluate models using a strict historical-training and future-testing protocol.
- We design a dynamic behavioral feature pipeline that captures 87 runtime behavioral tags as ordered event sequences and complementary aggregate statistics.
- We quantify behavioral drift between KronoDroid and recent AndroZoo samples using distributional metrics, including KL divergence, Jensen–Shannon divergence, entropy difference, and feature-prevalence change.
- We analyze which runtime behaviors increase or decrease across time, enabling interpretable discussion of why models trained on historical samples may fail on recent malware.
- We compare sequential and hybrid learning models, including LSTM, BiLSTM, Transformer, and Transformer–BiLSTM variants, under a deployment-oriented temporal split.

The remainder of this paper is organized as follows. Section II reviews related work on Android malware detection, realistic temporal evaluation, concept drift adaptation, and behavioral drift analysis. Section III details the dynamic analysis and feature extraction methodology. Subsequent sections describe the datasets, experimental setup, results, and implications for long-term Android malware detection.

## II. RELATED WORK

Android malware detection has evolved from static feature classification toward dynamic, hybrid, and sequence-based learning. However, the central question for a deployable detector is no longer only which representation achieves the highest score on a fixed benchmark. A detector must also remain reliable when applications, Android APIs, benign software practices, and malware behavior change over time. This section therefore reviews prior work around four connected themes: Android malware detection paradigms, temporal bias and realistic evaluation, drift adaptation, and behavioral drift interpretation. Across these themes, we emphasize a recurring limitation: much of the literature either improves detection accuracy, constrains evaluation, or adapts models after degradation, but does not explain the behavioral changes that make future samples difficult to classify.

### A. Android Malware Detection Paradigms

Static analysis remains attractive because permissions, manifest entries, API calls, intents, and bytecode features can be extracted without executing an application. Permission- and API-based models have been studied with classical classifiers such as SVM, KNN, Naive Bayes, random forests, and other machine learning algorithms [2]–[4], [18]. Static methods scale well and are suitable for pre-installation screening, but their evidence can be weakened by obfuscation, packing, API evolution, and changes in Android development practices.

Dynamic analysis shifts attention from declared or disassembled artifacts to observed behavior. Runtime system calls, API traces, network traffic, and resource interactions can

expose behaviors that static inspection misses [5]–[8]. Recurrent, attention-based, and transformer-based models further exploit temporal ordering by treating API or behavioral logs as sequences suitable for representation learning [19]–[23]. Hybrid approaches combine static and dynamic modalities, often improving benchmark performance by joining complementary signals [9]–[11]. These advances show that richer representations can improve discrimination under conventional evaluation, but they usually treat the dataset as a fixed population. As a result, they leave open whether dynamic and sequence-based signals remain stable when the test applications are collected years after the training data, which is the setting faced by real deployed detectors.

### B. Temporal Bias and Realistic Evaluation

The temporal dimension has long been recognized as a threat to Android malware evaluation. Allix et al. [12] showed that evaluation protocols must account for time, because models trained and tested on temporally mixed data can misrepresent real-world generalization. TESSERACT made this concern central by formalizing spatial and temporal constraints for malware-classifier evaluation and showing that random or unrealistic splits can inflate performance [13]. Its extended analysis further reinforces that high reported F1-scores do not settle malware classification when models decay under temporal and spatial shifts [14]. These studies are important because they transform temporal ordering from a dataset detail into an evaluation requirement.

However, realistic evaluation alone does not explain the cause of failure. A detector that performs well under random splitting may have learned correlations that are unavailable at deployment time, while a detector evaluated with a strict chronological split must classify applications that did not exist when the model was trained. KronoDroid supports this style of analysis by providing a timestamped Android dataset with static and dynamic features over a long historical period [15]. Recent time-stamped and self-supervised Android malware studies similarly argue that timestamp verification and time-aware evaluation are necessary for credible robustness claims [24]. The recent review by Guerra-Manzanares [16] synthesizes these concerns and argues that Android malware detection remains open, highlighting neglected concept drift, outdated datasets, inconsistent dynamic data, model security, and limited explainability. Thus, the evaluation literature convincingly shows that temporal bias exists, but it does not fully characterize which runtime behaviors shift and how those shifts affect dynamic malware detection.

### C. Concept Drift Handling through Online and Active Learning

Several studies address temporal degradation by updating the detector over time. DroidOL introduced online learning for Android malware detection, using graph-kernel features and a passive-aggressive classifier to adapt to malware population drift [25]. CASANDRA extended this idea with context-aware dependence-graph features and online learning, emphasizing both adaptiveness and explanatory feature evidence [26].

AndroDet applied online learning to Android obfuscation detection, showing how adaptive models can identify evolving obfuscation techniques after observing relatively few samples [27]. DroidEvolver further pursued self-evolving Android malware detection by updating feature knowledge and pseudo-labels without relying on fully labeled future samples [28]. Together, these methods treat drift as a learning problem: when the distribution changes, the model must be refreshed.

Other work treats drift as an uncertainty or maintenance problem. TRANSCEND detects aging malware classifiers during deployment by comparing new samples with the training distribution, warning before sustained performance degradation occurs [29]. TRANSCEND revisits this idea through conformal rejection, allowing uncertain samples under drift to be quarantined for expert analysis rather than forced into unreliable predictions [30]. Recent Android-specific studies continue this direction with active learning and streaming evaluation. Guerra-Manzanares and Bahsi [31] examine how feature space, balancing, timestamp choice, and noisy labels affect long-term Android malware detection with active learning. ActDroid frames Android malware detection as an active online learning problem with label delay and drift-aware retraining [32]. These approaches are valuable for maintaining deployed detectors, yet they mainly decide when and how a model should be updated. They do not directly answer what changed in runtime behavior, which behavioral categories became more or less prevalent, or whether sequence-level dynamics themselves drifted.

#### D. Representation Robustness and Behavioral Drift Analysis

A complementary line of research asks how to learn features that remain stable across time. APIGraph uses API semantics to reduce model aging by mapping API-level evidence into semantic relations that can better survive Android framework and malware evolution [33]. TIF explicitly targets temporal invariance, arguing that empirical-risk-minimization detectors can rely on unstable discriminative features and therefore degrade under temporal drift [34]. Self-supervised Android malware detection with BYOL similarly seeks more robust representations while retaining timestamp-aware evaluation [24]. These works move beyond simple retraining by trying to learn representations that are less sensitive to time. Nevertheless, robustness is still treated mainly as an optimization objective; the behavioral meaning of the drift remains underexplored.

Forensic drift analysis provides the missing interpretive layer. Zola et al. [35] show that temporal distribution shifts should be dissected through rolling evaluation, temporal aggregation, misclassification analysis, and feature inspection, because simply reporting performance decay does not explain why the decay occurred. This perspective directly motivates our study. Instead of only training a new dynamic model, we measure how Android runtime behavior changes from historical KronoDroid samples to recent AndroZoo samples. By comparing tag distributions, category frequencies, sequence lengths, entropy, and concrete prevalence changes, the proposed work connects temporal evaluation with behavioral explanation. The gap addressed here is therefore the absence

of a dynamic, sequence-aware Android malware study that jointly evaluates future generalization and explains the runtime behavioral drift responsible for that generalization challenge.

### III. MATERIALS AND METHODS

#### A. Dynamic Analysis Infrastructure

Dynamic behavioral analysis of Android applications requires a controlled, reproducible execution environment that can process thousands of APKs at scale while maintaining clean system state between samples. We designed a distributed analysis pipeline built around four parallel Genymotion Android 8.0 emulator instances, each running ARM Translation support to faithfully emulate real-device behavior. The pipeline is composed of three tightly integrated layers: a shell-level orchestration framework, a Python-based analysis coordinator, and a Frida TypeScript instrumentation agent injected into the target process.

#### B. Parallel Emulator Orchestration

Unlike prior dynamic malware studies that primarily focus on sequence modeling over sandbox logs or summarized behavioral traces, our protocol emphasizes controlled, repeatable, and fault-tolerant behavioral acquisition at the per-application level, ensuring that downstream learning is built on stable and comparable runtime observations rather than execution artifacts or collection noise. [20], [21], [23] To enable scalable and isolated dynamic analysis, we employ a parallelized emulator orchestration strategy based on multiple independent Android virtual devices. Each emulator instance operates as an isolated execution environment, allowing concurrent processing of applications while preventing cross-sample contamination. Unlike single-device analysis pipelines commonly used in prior work, this design significantly improves throughput while preserving behavioral fidelity. At runtime, applications are distributed across the emulator pool, enabling parallel execution and efficient utilization of computational resources. A key challenge in large-scale dynamic analysis is the degradation of emulator performance over long-running sessions due to accumulated system state, including residual processes, memory fragmentation, and cached runtime artifacts. To address this limitation, we incorporate periodic environment resetting, ensuring that each emulator is regularly restored to a clean and stable state. This strategy maintains consistent execution conditions across samples and mitigates performance drift. To further ensure robustness, the orchestration framework continuously monitors emulator health and automatically recovers unresponsive instances without interrupting the overall analysis process. This fault-tolerant design prevents pipeline stalls and enables uninterrupted large-scale data collection.

Finally, to support distributed processing, we implement a synchronization mechanism that coordinates APK assignment across parallel workers. This prevents duplicate analysis and ensures that each application is processed exactly once. Additionally, a checkpoint-based logging mechanism allows the system to resume from the last completed state in case of interruption, eliminating redundant computation.

### C. Per-APK Analysis Protocol

For each application, we employ a standardized and fully deterministic analysis protocol designed to ensure reproducibility, behavioral consistency, and isolation across samples. Prior to execution, the emulator environment is restored to a clean baseline state. This guarantees that each application is analyzed under identical system conditions, eliminating residual artifacts such as background processes, cached data, or previously injected instrumentation that could otherwise introduce bias or cross-sample contamination. Following environment initialization, the target application is installed and executed using a controlled launch procedure. To maximize behavioral coverage, the framework supports both explicit activity invocation and fallback event-driven stimulation, enabling the execution of applications that do not expose standard entry points—a common characteristic of evasive or malicious samples. To monitor runtime behavior, we adopt a dynamic instrumentation strategy based on process-level attachment. Unlike spawn-based approaches that require modified applications or specialized runtime environments, our method operates on unmodified APKs within standard emulator configurations. This design choice improves compatibility and allows the analysis of real-world samples without prior repackaging or instrumentation embedding. Instrumentation is deployed through a precompiled agent that is reused across analysis sessions, minimizing per-sample overhead and ensuring consistent monitoring behavior. During execution, behavioral data is continuously collected through a controlled polling mechanism over a fixed analysis window. This approach captures both transient and sustained runtime activities while maintaining a bounded and comparable observation period across all samples. To address instability caused by non-responsive applications or deliberate stalling techniques (e.g., anti-analysis delays), the protocol incorporates timeout-aware monitoring and early termination criteria. This prevents indefinite blocking and ensures that the pipeline remains responsive under adversarial conditions. Upon completion of the execution window, the collected behavioral data is consolidated into a structured feature representation and stored atomically. The application is then removed from the environment, and the system proceeds to the next sample. Overall, this protocol ensures that each application is analyzed under consistent, controlled, and fault-tolerant conditions, enabling reliable large-scale behavioral data collection while remaining resilient to common evasion and instability issues observed in dynamic malware analysis.

### D. API Instrumentation and Feature Extraction

1) *Frida Instrumentation Agent*: Unlike prior work that models malware predominantly through raw API call sequences, filtered behavioral reports, or frequency-oriented API representations, our instrumentation layer is explicitly behavior-centric: it groups runtime events into semantically defined security-relevant categories and preserves their frequency, timing, and ordering simultaneously [21], [22]. To capture fine-grained runtime behavior, we design a behavior-centric instrumentation agent based on dynamic API hooking.

The agent monitors a diverse set of Android API entry points spanning multiple behavioral categories, including dynamic code loading, network communication, cryptographic operations, file system access, inter-process communication, and user privacy-sensitive actions, as summarized in Table I.

TABLE I  
BEHAVIORAL CATEGORIES OF THE INSTRUMENTED API HOOKS

| Category                     | Hook Points |
|------------------------------|-------------|
| Reflection & Dynamic Loading | 8           |
| Network Communication        | 7           |
| Cryptography                 | 9           |
| Surveillance & Privacy       | 8           |
| File Operations              | 6           |
| Command Execution            | 3           |
| Persistence Mechanisms       | 5           |
| Benign SDK Signals           | 10          |
| Anti-Analysis Evasion        | 7           |
| Package Management           | 3           |
| Inter-Process Communication  | 4           |
| Clipboard & Overlay UI       | 3           |
| Accessibility Services       | 4           |
| Screen & Input Injection     | 2           |
| Advanced Attack Vectors      | 6           |
| Push & Remote Configuration  | 2           |
| Total                        | 87          |

Unlike prior approaches that rely solely on coarse-grained features (e.g., aggregate system calls or network statistics), our instrumentation strategy explicitly models semantically meaningful behavioral groups. This enables the system to distinguish between functionally similar but contextually different activities, such as benign analytics logging versus privacy-invasive data collection. While several recent transformer-based studies improve dynamic malware analysis through preprocessing, token normalization, or sequence classification, they generally remain focused on distinguishing malicious semantics from raw behavioral content; in contrast, we explicitly model benign SDK signals as a separate category to reduce confusion between legitimate analytics/advertising behavior and privacy-invasive activity with superficially similar API footprints [21], [23], [36]. Each intercepted API call contributes to three complementary representations. First, global counters track the frequency of behavior across categories. Second, first-occurrence timestamps capture temporal characteristics, enabling the analysis of early-stage execution patterns. Third, all events are recorded in a structured sequential log, preserving ordering information for downstream sequence modeling. To ensure robustness, instrumentation is designed to be non-intrusive and fault-tolerant: failures in individual hook points do not interrupt the analysis process. This guarantees stable data collection even in the presence of incomplete class loading or obfuscated code structures.

2) *Burst Behavior Modeling*: Traditional frequency-based features fail to capture short-lived but high-intensity execution phases commonly observed in malware, such as payload unpacking or rapid data exfiltration. To address this limitation, we introduce a burst-based feature that measures the peak density of API activity within a sliding temporal window. This feature provides a compact representation of execution intensity, enabling the detection of transient but highly indicative behavioral spikes.

3) *Data Export*: The instrumentation agent outputs a unified behavioral representation combining frequency-based, temporal, and sequential information. This includes aggregated event counts, timing features, burst characteristics, and the ordered API event sequence, forming the foundation for the downstream feature engineering pipeline.

#### E. Sequence Log Throttling

Unlike approaches that rely on raw execution logs, full JSON reports, or unthrottled API sequences, our sequence construction deliberately controls event dominance so that frequent but low-value behaviors do not suppress rare precursor events that are often more diagnostic of multi-stage malicious activity [23], [36]. A key challenge in sequence-based dynamic analysis is the imbalance between high-frequency and low-frequency behavioral events. In practice, a small subset of APIs—such as reflection or network communication can dominate execution traces, rapidly saturating the sequence log and obscuring rare but security-critical events. Naïve logging strategies that rely on a global event limit tend to over-represent frequent but often benign activities, while under-representing infrequent yet highly discriminative behaviors (e.g., privilege escalation, accessibility abuse, or command execution). This imbalance degrades the effectiveness of downstream sequence models, particularly attention-based architectures that rely on diverse contextual signals. To address this limitation, we introduce a category-aware throttling mechanism that enforces per-event-type constraints during sequence construction. Rather than allowing dominant behaviors to monopolize the log, this approach ensures that multiple behavioral categories are proportionally represented within each execution trace. Importantly, this design preserves rare but diagnostically significant events while preventing noise saturation caused by repetitive API calls. As a result, the resulting sequence representation maintains both diversity and discriminative power, improving the model’s ability to capture meaningful behavioral patterns across applications. This design is particularly important for capturing multi-stage attack patterns, where rare precursor events may otherwise be lost in dense execution traces.

#### F. Feature Engineering Pipeline

Unlike prior studies that predominantly operate on a single behavioral view—such as API sequences, filtered execution reports, or frequency-based representations—our feature engineering pipeline integrates four complementary views of runtime behavior: direct event intensity, higher-order derived semantics, temporal execution dynamics, and explicit sequence-chain abstractions [21]–[23]. Raw outputs produced by the instrumentation agent are transformed into a structured tabular representation through a multi-stage feature engineering pipeline. The resulting feature vector consists of 168 dimensions organized into four complementary groups: raw behavioral counters, derived features, temporal features, and sequence-based features (Table II). This design aims to capture not only the presence of behavioral signals, but also their intensity, temporal characteristics, and contextual relationships.

Unlike prior work that relies primarily on single-view features (e.g., frequency-only or sequence-only), our pipeline integrates multiple behavioral perspectives into a unified representation.

TABLE II  
FEATURE SCHEMA OF THE EXTRACTED BEHAVIORAL DATASET

| Group                    | Sub-group                          | Count      |
|--------------------------|------------------------------------|------------|
| Meta                     | Package name, label, timestamp     | 3          |
| Raw Counters             | Reflection & dynamic loading       | 5          |
| Raw Counters             | Cryptography                       | 5          |
| Raw Counters             | Anti-analysis                      | 7          |
| Raw Counters             | General behavior                   | 2          |
| Raw Counters             | Intent & IPC                       | 6          |
| Raw Counters             | File system                        | 6          |
| Raw Counters             | Network & socket                   | 5          |
| Raw Counters             | Persistence & privilege            | 5          |
| Raw Counters             | Process & memory                   | 5          |
| Raw Counters             | Accessibility, overlay & clipboard | 4          |
| Raw Counters             | Telephony & device identity        | 4          |
| Raw Counters             | Extended v4 counters               | 25         |
| <b>Raw Counters</b>      | <b>Sub-total</b>                   | <b>79</b>  |
| Derived Features         | Composite behavioral scores        | 8          |
| Derived Features         | Behavioral ratios                  | 5          |
| Derived Features         | Boolean pattern flags              | 7          |
| Derived Features         | Session-normalized scores          | 4          |
| Derived Features         | Per-second rate features           | 6          |
| Derived Features         | LogIp outlier-stable features      | 6          |
| <b>Derived Features</b>  | <b>Sub-total</b>                   | <b>36</b>  |
| Temporal Features        | First-occurrence timestamps        | 8          |
| Temporal Features        | Burst & session duration           | 2          |
| Temporal Features        | Temporal ordering flags            | 4          |
| Temporal Features        | Early-5s activity flags            | 3          |
| Temporal Features        | Inter-event deltas                 | 4          |
| Temporal Features        | Session anomaly flag               | 1          |
| <b>Temporal Features</b> | <b>Sub-total</b>                   | <b>22</b>  |
| Sequence Features        | Binary chain presence flags        | 8          |
| Sequence Features        | Chain statistics                   | 2          |
| Sequence Features        | Chain occurrence counts            | 5          |
| Sequence Features        | First chain completion times       | 5          |
| Sequence Features        | Attack vector chain flags & counts | 4          |
| Sequence Features        | Attack vector chain timings        | 4          |
| <b>Sequence Features</b> | <b>Sub-total</b>                   | <b>28</b>  |
| <b>Total</b>             |                                    | <b>168</b> |

1) *Raw Behavioral Counters*: The first group consists of direct API call frequencies aggregated across all monitored behavioral categories (Table II). These features provide a coarse-grained view of application activity, capturing the overall intensity of operations such as dynamic code loading, cryptographic usage, network communication, file system interaction, and privacy-sensitive actions. While informative, raw counts alone are insufficient for distinguishing benign and malicious applications due to their sensitivity to execution duration and application complexity. Therefore, they serve as a foundational layer upon which more discriminative features are constructed.

2) *Derived Behavioral Features*: To overcome the limitations of raw frequency signals, we construct a set of derived features that capture higher-level behavioral semantics (Table II). These include composite behavioral scores, inter-category ratios, Boolean pattern indicators, and normalized activity measures. Composite scores aggregate related API calls into interpretable behavioral dimensions such as network activity, persistence, stealth, and exfiltration. To mitigate bias introduced by varying execution lengths, selected features are normalized by session duration, enabling fair comparison

across applications. Behavioral ratios further capture relationships between categories, providing invariance to absolute activity levels. In addition, Boolean pattern flags encode compound behavioral signatures that are difficult to capture through individual features alone, such as the co-occurrence of encoding and network transmission or the combination of privilege escalation and dynamic execution. This multi-view design is motivated by the limitations of raw-count and sequence-only modeling reported in prior work: frequency-based approaches may ignore temporal context, whereas pure sequence models can underutilize stable aggregate behavioral evidence unless it is explicitly encoded. [21], [37]. Finally, rate-based and log-transformed features are introduced to stabilize high-variance distributions and improve robustness for gradient-based learning models.

3) *Temporal Features*: The third group captures the temporal dynamics of application behavior (Table II). These features are derived from the timing of first-occurrence events and encode when specific behaviors emerge during execution. Early-stage activity is particularly informative in malware detection, as malicious applications often trigger payload execution shortly after launch. To capture this phenomenon, we include early-activity indicators, inter-event timing relationships, and ordering constraints between behavioral categories. Additionally, session-level indicators such as burst intensity and execution duration are incorporated to detect anomalous runtime patterns, including stalling behavior or delayed execution used to evade sandbox analysis.

4) *Sequence-Based Behavioral Features*: The final group extracts higher-order behavioral patterns from the sequential API event log (Table II). Rather than relying solely on token sequences for downstream models, we explicitly encode structured interaction patterns between events. These features capture ordered co-occurrence relationships, such as data access followed by network transmission or persistence setup preceding execution. Such patterns are particularly indicative of multi-stage attack workflows and cannot be effectively represented through isolated counters or timestamps. We further incorporate chain-level statistics, including occurrence counts and timing of first appearance, enabling the model to reason about both the presence and progression of complex behavioral sequences.

5) *Discriminative Feature Validation*: To verify the effectiveness of the proposed feature set, we analyze the distribution of selected features across benign and malicious samples (Table III). Notably, features such as write-to-read ratios, file-to-network behavioral chains, and early-stage communication indicators exhibit strong separation between classes, supporting the discriminative capacity of the engineered representation.

### G. Dual-Format Output

The pipeline generates two complementary output formats per analysis run to preserve both structured summaries and fine-grained behavioral dynamics.

1) *Tabular CSV*: Each APK is represented as a fixed-length vector of 168 features, ensuring that all successfully instrumented applications are included regardless of execution

TABLE III  
DISTRIBUTION OF THE SELECTED FEATURES ACROSS LABELS

| Feature                 | Group                      | Malware | Benign | $\Delta$ |
|-------------------------|----------------------------|---------|--------|----------|
| network_score_per_sec   | Derived / Normalized Score |         |        |          |
| crypto_to_network_ratio | Derived / Ratio            |         |        |          |
| has_exfil_pattern       | Derived / Boolean Flag     |         |        |          |
| has_evasion_pattern     | Derived / Boolean Flag     |         |        |          |
| early_exec_flag         | Temporal / Binary          |         |        |          |
| early_network_flag      | Temporal / Binary          |         |        |          |
| crypto_before_network   | Temporal / Binary          |         |        |          |

length. This representation captures stable aggregate, temporal, and derived behavioral characteristics.

2) *Sequence JSONL*: In parallel, a sequence-based representation is constructed, preserving the ordered API event log along with session metadata. This format supports sequence-based models, particularly transformer architectures that rely on contextual relationships between events. Only applications producing at least 10 API events are included in the sequence dataset. Short execution traces are excluded as they lack sufficient context for attention mechanisms to learn meaningful dependencies, a limitation observed in prior sequence-based malware detection approaches [20], [37]. Importantly, this is a modeling decision rather than data exclusion, as all samples remain fully represented in the tabular feature space. This dual-format design addresses a key limitation of prior work, which typically relies on a single representation paradigm. By preserving both tabular and sequential views, the pipeline enables complementary modeling of global behavioral patterns and temporal dependencies.

## IV. PROPOSED DUAL-STREAM DETECTION ARCHITECTURE

To fully exploit the complementary nature of our two output modalities tabular behavioral profiles capturing aggregate statistics and Boolean patterns, and sequential API call logs capturing temporal ordering and behavioral chains we propose KangalGuard, a dual-stream detection architecture that integrating tabular behavioral profiles and sequential API logs into a unified framework for classifying Android behaviors as malware or benign (Fig. 1).

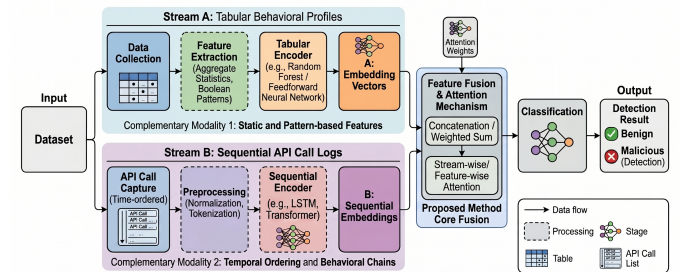


Fig. 1. Proposed Method: Dual Stream Detection Architecture

1) *Sequence Stream: API-Call Transformer Encoder*: The sequence stream models the ordered API event log as a discrete token sequence, where each event corresponds to one of 78 API categories. Unlike prior sequence-based approaches that

rely solely on token order, we augment token embeddings with continuous temporal position encodings representing the millisecond offset from session start. This enables the model to capture not only the ordering of events but also their temporal density, which is critical for distinguishing behaviors such as rapid initialization bursts versus distributed activity patterns.

The resulting sequence is processed using a Transformer encoder architecture, which is well-suited for modeling long-range dependencies and contextual relationships in execution traces. In contrast to recurrent models that struggle with long sequences, attention-based architectures can effectively capture both local interactions and global behavioral structure across API events.

The final representation is obtained from the [CLS] token embedding at the top layer, producing a fixed-dimensional sequence vector

$$z_{\text{seq}} \in \mathbb{R}^{d_{\text{model}}}. \quad (1)$$

To ensure robustness, applications with insufficient sequence information (fewer than 10 events) are handled through a masked sequence pathway. Instead of discarding such samples or applying heuristic padding, we introduce a learned fallback embedding that allows the model to operate in a tabular-only mode when sequence data is unavailable. This design ensures consistent inference behavior and avoids bias introduced by incomplete or noisy execution traces.

2) *Tabular Stream: Feature MLP*: The tabular stream processes the 168-dimensional feature vector using a multi-layer perceptron designed to capture stable, aggregated behavioral patterns. Unlike the sequence stream, which focuses on temporal ordering and contextual dependencies, this branch models global behavioral characteristics derived from frequency, ratio, and temporal summary features.

The network consists of fully connected layers with batch normalization and residual connections, producing a compact embedding

$$z_{\text{tab}} \in \mathbb{R}^{d_{\text{tab}}}. \quad (2)$$

Residual connections are employed to facilitate gradient flow and preserve informative feature interactions, particularly in the presence of heterogeneous feature groups.

Input features are standardized per column using training-set statistics to ensure consistent scaling across dimensions and improve optimization stability. Let the input feature vector be denoted as  $x \in \mathbb{R}^{168}$ . Each feature dimension is standardized as

$$\tilde{x}_i = \frac{x_i - \mu_i}{\sigma_i}, \quad (3)$$

where  $\mu_i$  and  $\sigma_i$  denote the mean and standard deviation of the  $i$ -th feature computed over the training set.

This is particularly important given the mixture of raw counts, normalized scores, and transformed features within the input space.

This stream serves as a complementary representation to the sequence encoder by preserving information that may be diluted in token-based sequence modeling, such as global activity intensity, cross-category relationships, and session-level behavioral summaries.

3) *Fusion and Classification*: The two stream embeddings are fused through a joint representation layer that combines complementary behavioral information from both modalities. Specifically, the sequence embedding  $z_{\text{seq}}$  and tabular embedding  $z_{\text{tab}}$  are concatenated and normalized before classification:

$$\hat{y} = \sigma(W_c \cdot \text{LayerNorm}([z_{\text{seq}}; z_{\text{tab}}]) + b_c), \quad (4)$$

where  $[z_{\text{seq}}; z_{\text{tab}}]$  denotes vector concatenation,  $W_c$  and  $b_c$  are learnable classification parameters, and  $\sigma(\cdot)$  represents the sigmoid activation function.

While simple in form, this fusion strategy is motivated by the complementary nature of the two representations: the sequence stream captures temporal dependencies and contextual ordering, whereas the tabular stream preserves stable aggregate and cross-category behavioral signals. Joint normalization ensures balanced contribution from both modalities, preventing dominance of either representation during training.

The model is trained end-to-end using binary cross-entropy loss:

$$\mathcal{L}_{\text{BCE}} = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]. \quad (5)$$

To further improve robustness under class imbalance and hard-to-classify samples, a focal loss variant with focusing parameter  $\gamma = 2$  is employed:

$$\mathcal{L}_{\text{focal}} = -[y(1 - \hat{y})^\gamma \log(\hat{y}) + (1 - y)\hat{y}^\gamma \log(1 - \hat{y})]. \quad (6)$$

4) *Baseline Models*: To rigorously evaluate the effectiveness of the proposed dual-stream architecture, we compare the method proposed in this study against a diverse set of baseline models under a controlled experimental setting. All models are trained and evaluated using identical data splits and input feature representations to ensure a fair comparison.

TABLE IV  
EVALUATED BASELINE MODELS AND THEIR CONFIGURATIONS.

| Model                  | Input         | Architecture                             |
|------------------------|---------------|------------------------------------------|
| LSTM                   | Sequence only | 2-layer LSTM + linear head               |
| BiLSTM                 | Sequence only | 2-layer bidirectional LSTM + linear head |
| Transformer-Only       | Sequence only | Encoder identical to sequence stream     |
| MLP-Tabular            | Tabular only  | MLP identical to tabular stream          |
| Random Forest          | Tabular only  | 500 trees, Gini criterion                |
| XGBoost                | Tabular only  | 500 estimators, learning rate 0.05       |
| <b>Proposed Method</b> | Both          | Dual-stream fusion (this study)          |

As shown in the Table IV, the selected baselines are designed to isolate and quantify the contribution of each modality and architectural component. The LSTM and BiLSTM models represent widely adopted recurrent architectures in dynamic malware analysis, particularly in recent hybrid and cascade-based frameworks [38], enabling a direct comparison between attention-based and recurrent sequence modeling paradigms.

The Transformer-Only baseline shares the same encoder design as the sequence stream of the proposed architecture, thereby isolating the effect of multi-modal fusion. Similarly, the MLP-Tabular baseline evaluates the discriminative capacity of aggregated behavioral features in the absence of temporal modeling.

Tree-based models, including Random Forest and XGBoost, are incorporated as strong classical baselines for tabular data,



providing a robust non-deep learning reference point. Finally, the comparison between the Transformer-Only baseline and the full model explicitly quantifies the contribution of the tabular modality, while the tabular-only baselines establish the upper bound achievable without sequential behavioral information.

## V. DATASETS

As shown in Table V, we evaluate the proposed framework on a carefully curated dataset comprising 30,000 samples from KronoDroid [15] (15,000 benign and 15,000 malware) and 6,000 recently collected samples from AndroZoo [17] (3,000 benign and 3,000 malware), dated to 2025–2026 and reserved exclusively for temporal shift evaluation.

TABLE V  
DATASET COMPOSITION AND SEQUENCE AVAILABILITY ACROSS MODALITIES.

| Dataset    | Tabular Data |         |       | Sequential Data |         |       |
|------------|--------------|---------|-------|-----------------|---------|-------|
|            | Benign       | Malware | Total | Benign          | Malware | Total |
| KronoDroid | 15000        | 15000   | 30000 | 6000            | 6000    | 12000 |
| AndroZoo   | 3000         | 3000    | 6000  | 1500            | 1500    | 3000  |

KronoDroid provides temporally diverse historical samples spanning a broad period of Android ecosystem evolution, whereas the AndroZoo split introduces more recent benign and malicious applications that better reflect contemporary SDK usage patterns and current malware behaviors. The 2026 subset is intentionally withheld from training and validation and is used only for testing, enabling a more realistic assessment of temporal generalization under distribution shift.

To prevent data leakage, we perform cross-source deduplication before feature extraction and model training. In particular, APKs collected from KronoDroid and AndroZoo are compared using their SHA-256 hashes, and duplicate samples are removed prior to split construction. We additionally verify package identifiers to detect repeated applications with different collection metadata. This procedure ensures that no identical APK appears in both the training/validation pool and the held-out temporal evaluation set. A lightweight Python-based preprocessing script is used to enforce this uniqueness constraint and to generate the final leakage-free sample lists.

All APKs undergo dynamic analysis as described in Sections III-C and III-D. Samples that fail to install, fail to produce a valid PID within the retry window, or trigger three consecutive RPC timeouts are logged to `failed_apks.csv` and excluded from the final dataset. Deduplication and exclusion are completed before model fitting to ensure that downstream evaluation reflects only valid and unique applications.

This setup enables a realistic evaluation scenario in which the model must generalize across temporal distribution shifts, rather than relying on randomly shuffled datasets that may overestimate real-world performance. Due to execution variability, sequence data is available for only a subset of applications, resulting in approximately 40% sequence availability for KronoDroid and 50% for AndroZoo. This variability further motivates the proposed dual-stream architecture, which

remains applicable even when sequence traces are incomplete or unavailable.

## VI. EXPERIMENTAL SETUP

Your setup goes here.

## VII. RESULTS AND DISCUSSION

Your results go here.

## VIII. CONCLUSION

Your conclusion goes here.

## REFERENCES

- [1] Kaspersky, “Mobile malware evolution 2023,” Kaspersky SecureList Report, 2023, available: <https://securelist.com/mobile-malware-evolution-2023/111681/>.
- [2] A. S. Shatnawi, Q. Yassen, and A. Yateem, “An android malware detection approach based on static feature analysis using machine learning algorithms,” *Procedia Computer Science*, vol. 201, pp. 653–658, 1 2022. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1877050922004999>
- [3] A. Pathak, T. S. Kumar, and U. Barman, “Static analysis framework for permission-based dataset generation and android malware detection using machine learning,” *EURASIP Journal on Information Security* 2024 2024:1, vol. 2024, pp. 33–, 10 2024. [Online]. Available: <https://link.springer.com/article/10.1186/s13635-024-00182-3>
- [4] J. Pang and J. Bian, “Android malware detection based on naive bayes,” *Proceedings of the IEEE International Conference on Software Engineering and Service Sciences, ICSESS*, vol. 2019-October, pp. 483–486, 10 2019. [Online]. Available: <https://ieeexplore.ieee.org/document/9040796>
- [5] G. Sathyadevi, J. Abishek, and B. S. Shakthieswaran, “Dynashield: android malware detection using dynamic analysis of network traffic,” *2024 International Conference on System, Computation, Automation and Networking, ICSCAN 2024*, 2024. [Online]. Available: <https://ieeexplore.ieee.org/document/10894660>
- [6] G. Ciaramella, F. Mercaldo, and A. Santone, “Dynamic analysis for explainable fine-grained android malware detection,” pp. 110–127, 2025. [Online]. Available: [https://link.springer.com/chapter/10.1007/978-3-031-76371-7\\_8](https://link.springer.com/chapter/10.1007/978-3-031-76371-7_8)
- [7] E. G. V. Enriquez and J. Gutiérrez-Cárdenas, “Dynamic malware analysis using machine learning-based detection algorithms,” *Interfases*, pp. 119–138, 7 2024.
- [8] A. Singh, M. Tanha, Y. Girdhar, and A. Hunter, “Interpretable android malware detection based on dynamic analysis,” in *International Conference on Information Systems Security and Privacy*, vol. 1. Science and Technology Publications, Lda, 2024, pp. 195–202.
- [9] M. U. Rashid, S. Qureshi, A. Abid, S. S. Alqahtany, A. Alqazzaz, M. ul Hassan, M. S. A. Reshan, and A. Shaikh, “Hybrid android malware detection and classification using deep neural networks,” *International Journal of Computational Intelligence Systems* 2025 18:1, vol. 18, pp. 52–, 3 2025. [Online]. Available: <https://link.springer.com/article/10.1007/s44196-025-00783-x>
- [10] K. Kılıç, İsmail Atacak, and İbrahim Alper Doğru, “Fabldroid: Malware detection based on hybrid analysis with factor analysis and broad learning methods for android applications,” *Engineering Science and Technology, an International Journal*, vol. 62, p. 101945, 2 2025. [Online]. Available: <https://doi.org/10.1109/access.2020.3006143>
- [11] A. Arora, “H-pintdroid: Hybrid android malware detection using permissions, intents and network traffic,” in *Conference Proceedings - 2025 4th International Conference on Advances in Computing, Communication, Embedded and Secure Systems, ACCESS 2025*. Institute of Electrical and Electronics Engineers Inc., 2025, pp. 481–486.
- [12] K. Allix, T. F. Bissyandé, Q. Jérôme, J. Klein, R. State, and Y. L. Traon, “Empirical assessment of machine learning-based malware detectors for android,” *Empirical Software Engineering*, vol. 21, no. 1, pp. 183–211, 2016.
- [13] F. Pendlebury, F. Pierazzi, R. Jordaney, J. Kinder, and L. Cavallaro, “TESSERACT: Eliminating experimental bias in malware classification across space and time,” in *Proceedings of the 28th USENIX Security Symposium*. USENIX Association, 2019, pp. 729–746.



- [14] Z. Kan, S. McFadden, D. Arp, F. Pendlebury, R. Jordaney, J. Kinder, F. Pierazzi, and L. Cavallaro, "TESSERACT: Eliminating experimental bias in malware classification across space and time (extended version)," *CoRR*, vol. abs/2402.01359, 2025. [Online]. Available: <https://arxiv.org/abs/2402.01359>
- [15] A. Guerra-Manzanares, H. Bahsi, and S. Nömm, "Kronodroid: Time-based hybrid-featured dataset for effective android malware detection and characterization," *Computers and Security*, vol. 110, 11 2021.
- [16] A. Guerra-Manzanares, "Machine learning for android malware detection: Mission accomplished? a comprehensive review of open challenges and future perspectives," 3 2024, *computers & Security*, vol. 138.
- [17] L. Li, J. Gao, M. Hurier, P. Kong, T. F. Bissyandé, A. Bartel, J. Klein, and Y. L. Traon, "Androzoo+: Collecting millions of android apps and their metadata for the research community," 9 2017. [Online]. Available: <http://arxiv.org/abs/1709.05281>
- [18] M. H. M. Saufan, S. Ahmad, Z. Zulkifli, N. H. Nasarudin, M. I. M. Ariff, and N. A. Sarkam, "Random forest-based approach to android malware detection in machine learning," *2024 5th International Conference on Artificial Intelligence and Data Sciences, AiDAS 2024 - Proceedings*, pp. 381–385, 2024. [Online]. Available: <https://ieeexplore.ieee.org/document/10730240>
- [19] H. D. Misalkar and P. Harshavardhanan, "Tdbamla: Temporal and dynamic behavior analysis in android malware using lstm and attention mechanisms," *Computer Standards and Interfaces*, vol. 92, 3 2025.
- [20] N. Youssef, N. Elbaraway, and A. Elmaghraby, "Transformer-based api call sequence modeling for dynamic malware detection," in *Conference Proceedings - IEEE SOUTHEASTCON*. Institute of Electrical and Electronics Engineers Inc., 2025, pp. 494–500.
- [21] N. Youssef, N. Elbarawi, and A. Elmaghraby, "Bert for dynamic malware detection: a sequence-based classification approach." Institute of Electrical and Electronics Engineers (IEEE), 1 2026, pp. 1–6.
- [22] L. Qian and L. Cong, "Channel features and api frequency-based transformer model for malware identification," *Sensors*, vol. 24, 1 2024.
- [23] D. Trizna, L. Demetrio, B. Biggio, and F. Roli, "Nebula: Self-attention for dynamic malware analysis," 10 2024. [Online]. Available: <http://arxiv.org/abs/2310.10664><http://dx.doi.org/10.1109/TIFS.2024.3409083>
- [24] A. Fu, H. Pei, and M. Tanha, "Self-supervised learning for android malware detection on a time-stamped dataset," *CoRR*, vol. abs/2604.23025, 2026. [Online]. Available: <https://arxiv.org/abs/2604.23025>
- [25] A. Narayanan, L. Yang, L. Chen, and J. Liu, "Adaptive and scalable android malware detection through online learning," in *Proceedings of the International Joint Conference on Neural Networks*, 2016.
- [26] A. Narayanan, M. Chandramohan, L. Chen, and Y. Liu, "Context-aware, adaptive, and scalable android malware detection through online learning," *IEEE Transactions on Emerging Topics in Computational Intelligence*, vol. 1, no. 3, pp. 157–175, 2017.
- [27] O. Mirzaei, J. M. de Fuentes, J. M. E. Tapiador, and L. González-Manzano, "ANDRODET: An adaptive android obfuscation detector," *Future Generation Computer Systems*, vol. 90, pp. 240–261, 2019.
- [28] K. Xu, Y. Li, R. H. Deng, K. Chen, and J. Xu, "DroidEvolver: Self-evolving android malware detection system," in *Proceedings of the 4th IEEE European Symposium on Security and Privacy*. IEEE, 2019, pp. 47–62.
- [29] R. Jordaney, K. Sharad, S. K. Dash, Z. Wang, D. Papini, I. Nourtdinov, and L. Cavallaro, "Transcend: Detecting concept drift in malware classification models," in *Proceedings of the 26th USENIX Security Symposium*. USENIX Association, 2017, pp. 625–642.
- [30] F. Barbero, F. Pendlebury, F. Pierazzi, and L. Cavallaro, "Transcending TRANSCEND: Revisiting malware classification in the presence of concept drift," in *Proceedings of the 2022 IEEE Symposium on Security and Privacy*. IEEE, 2022, pp. 805–823.
- [31] A. Guerra-Manzanares and H. Bahsi, "Experts still needed: Boosting long-term android malware detection with active learning," *Journal of Computer Virology and Hacking Techniques*, vol. 20, no. 4, pp. 901–918, 2024.
- [32] A. Muzaffar, H. R. Hassen, H. Zantout, and M. A. Lones, "ActDroid: An active learning framework for android malware detection," *Computers & Security*, vol. 160, 2026.
- [33] X. Zhang, Y. Zhang, M. Zhong, D. Ding, Y. Cao, Y. Zhang, M. Zhang, and M. Yang, "Enhancing state-of-the-art classifiers with api semantics to detect evolved android malware," in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. ACM, 2020, pp. 757–770.
- [34] X. Zheng, S. Yang, E. C. H. Ngai, S. Jana, and L. Cavallaro, "Learning temporal invariance in android malware detectors," *CoRR*, vol. abs/2502.05098, 2025. [Online]. Available: <https://arxiv.org/abs/2502.05098>
- [35] F. Zola, J. L. Bruse, and M. Galar, "Temporal analysis of distribution shifts in malware classification for digital forensics," in *Proceedings of the 8th IEEE European Symposium on Security and Privacy Workshops*. IEEE, 2023, pp. 439–450.
- [36] T. Quertier, B. Marais, G. Barrué, S. Morucci, S. Azé, and S. Salladin, "A lean transformer model for dynamic malware analysis and detection," 8 2024. [Online]. Available: <http://arxiv.org/abs/2408.02313>
- [37] A. K. J. N. D. Patel, D. Saravanan, and A. Patel, "Comparative study of natural language processing models for malware detection using api call sequences," 2 2026. [Online]. Available: <https://www.preprints.org/manuscript/202602.1359/v1>
- [38] B. B. Gupta, A. Gaurav, V. Arya, S. Bansal, R. W. Attar, A. Alhomoud, and K. Psannis, "Earthworm optimization algorithm based cascade lstm-gru model for android malware detection," *Cyber Security and Applications*, vol. 3, 12 2025.